

MIRC701 實驗室研究主題與計畫訪談紀錄表

訪談日期：115 年 6 月 日 時 分

論文名稱：Detecting and Preventing Input Injection in LLM-based Agents with 3-Layer Filtering
(基於大型語言模型代理之輸入注入攻擊偵測與防禦：三層過濾架構)

受訪者姓名：黎明朋 (LeMinhBang)

一、研究動機 (Motivation):

Q1: What problem are you trying to solve? (你想解決什麼問題?)

A1: I am trying to solve the input injection problem in LLM-based agents. In these systems, the agent does not only receive direct user prompts, but also consumes retrieved documents, tool outputs, and intermediate reasoning traces. Attackers can hide instructions in these channels and gradually redirect the agent's behavior, causing unauthorized tool execution or data leakage. My research addresses this by detecting whether the accumulated interaction history still aligns with the user's original intent.

Q2: Why is this problem important? (為什麼這個問題重要?)

A2: This problem is important because LLM-based agents can act on behalf of users and may have access to high-risk tools such as email, calendar, file search, financial services, or cybersecurity tools. If an attacker can manipulate the agent through indirect or multi-channel inputs, the consequence is no longer only a wrong text response; it can become a real action, such as sending private data or invoking an unauthorized tool. Therefore, securing the reasoning process of agents is necessary before they can be safely deployed in real environments.

Q3: Who will benefit if your research succeeds? (若研究成功，誰會受益?)

A3: The main beneficiaries are users, organizations, and developers who deploy LLM-based agents in real workflows. Users benefit because their private information and tool permissions are better protected. Organizations benefit because the defense reduces the risk of data leakage, policy violations, and unauthorized actions. Developers also benefit because InteractionGuard provides a modular defense layer that can be integrated into agent systems without completely redesigning the agent or removing its autonomy.

二、研究方法:

Q4: Why did you choose this method? (為什麼選擇這種方法?)

A4: I chose a three-layer cascaded method because the defense must balance security and efficiency. A single detector is usually biased: anomaly detection can catch unknown attacks but may produce false positives; supervised classification is accurate for known patterns but weaker on zero-day attacks; LLM auditing can reason over complex histories but is expensive. The cascade combines their strengths: Layer 1 quickly filters normal behavior, Layer 2 blocks known malicious patterns, and Layer 3 audits the remaining difficult cases semantically.

Q5 : What alternatives did you consider? (你考慮過哪些替代方法?)

A5: I considered several alternatives, including prompt-level filtering, RAG document filtering, tool-output checking, execution-level validation, and using only an LLM-based guardrail. However, prompt-level and RAG-level filters inspect isolated inputs and cannot reliably detect correlated attacks distributed across channels. Execution-level validation checks local actions but may miss the long-range reasoning path that caused the action. A pure LLM guardrail has better semantic ability, but the latency and cost are too high if every interaction is audited by an LLM.

Q6 : Why is your method better than existing approaches? (你的方法比現有方法好在哪裡?)

A6: The proposed method is better because it changes the defense perspective from isolated input inspection to interaction-history auditing. Existing approaches usually protect one channel, such as prompt, RAG, or tool output. In contrast, InteractionGuard observes the accumulated behavior of the agent, including prompt, retrieval, tool output, and agent response. This makes it more suitable for correlated attacks, where each fragment may look benign individually but becomes malicious when combined. The cascade also reduces LLM auditing cost by sending only unresolved cases to Layer 3.

Q7 : What assumptions does your model make? (你的模型有哪些假設?)

A7: The model assumes that the system prompt, defense pipeline, and model parameters are trusted and cannot be modified by the attacker. It treats external textual sources, including retrieved documents and tool outputs, as untrusted or partially trusted. It also assumes that the agent execution trace can be collected and converted into a structured interaction history with role tokens such as [PROMPT], [RETRIEVE], [TOOL], and [AGENT]. Finally, the current evaluation assumes text-based attacks and does not yet cover multimodal inputs or fully adaptive attackers with unlimited knowledge.

三、資料與實驗：

Q8 : How did you collect your dataset? (你的資料集如何取得?)

A8: I collected the dataset using a LangGraph-based ReAct agent testbed that generates complete execution traces under benign and adversarial scenarios. Benign traces come from normal prompts, clean RAG retrieval, and normal tool interactions. Malicious traces include prompt injection derived from public prompt-injection data, tool injection inspired by InjecAgent-style methodology, RAG poisoning based on PoisonedRAG-style document substitution, and correlated attacks constructed by distributing adversarial fragments across prompt, retrieval, and tool-output channels. The traces are split at the episode level to prevent data leakage, then expanded into interaction-history prefixes for detection evaluation.

Q9 : Why are these evaluation metrics appropriate? (為何選擇這些評估指標?)

A9: These metrics are appropriate because the defense has three goals: security, usability, and efficiency. Precision measures whether flagged malicious cases are truly malicious, which relates to false alarms. Recall measures how many attacks are detected, which is critical for security. F1-score

summarizes the balance between precision and recall. FPR and FNR show the trade-off between over-blocking benign interactions and missing attacks. Latency measures whether the defense is practical for real-time use. ASR is also necessary because it measures whether attacks still succeed at the episode level after passing the defense.

Q10 : How do you verify generalization ability? (如何驗證泛化能力?)

A10: I verify generalization ability in several ways within the current research scope. First, the dataset is split by complete episodes before prefix expansion, so training and testing prefixes do not come from the same execution episode. Second, the evaluation includes multiple attack categories, including prompt injection, tool injection, RAG poisoning, and correlated attacks. Third, I test different layer configurations, LLM backends, Layer 1 thresholds, and benign-heavy workloads. However, I do not claim universal generalization; broader benchmark evaluation, such as AgentDojo, remains an important future extension.

四、創新性 (Novelty):

Q11 : What is the novelty of your work? (你的研究創新點是什麼?)

A11: The novelty is the interaction-level defense paradigm for LLM-based agents. Instead of asking whether one prompt, one retrieved document, or one tool output is malicious, my work asks whether the accumulated interaction history still follows the user's original intent. Based on this idea, I propose InteractionGuard, a three-layer cascaded architecture combining benign-only anomaly detection, supervised transformer classification, and LLM-based semantic auditing. This combination is designed specifically for multi-channel and correlated attacks that emerge through reasoning-time aggregation.

Q12 : What is the biggest difference from previous studies? (與過去研究最大的差異是什麼?)

A12: The biggest difference from previous studies is the protected object. Previous defenses mainly protect isolated inputs or local tool actions. My work protects the agent's behavior over time by auditing the full interaction history. This is important because in agent systems, malicious behavior can emerge only after the agent combines prompt content, retrieved documents, tool outputs, and previous reasoning steps. Therefore, the defense is not only channel-specific, but behavior-aware and history-aware.

Q13 : Why has nobody solved this problem before? (為什麼以前沒有人解決?)

A13: It is not accurate to say that nobody has tried to solve agent security. Many studies have proposed prompt filters, RAG filters, tool validators, and LLM-based guardrails. The reason this specific problem remains difficult is that LLM agents combine multiple untrusted text sources inside one reasoning context, and the boundary between instruction and data is still ambiguous. Correlated attacks are especially hard because no single input may look clearly malicious. Solving this requires both a new perspective and a practical architecture that can reason over history without making the system too slow.

五、未來工作 (Future Work) :

Q14 : What are the limitations of your work? (研究限制是什麼?)

A14: The main limitations are the evaluation scope, attack diversity, and deployment cost. The current system is evaluated mainly on a self-built LangGraph testbed, so broader benchmark-grounded validation is still needed. The attacks are text-based and cover representative categories, but they cannot cover all future adaptive or multimodal attacks. Layer 3 also depends on an LLM auditor, so its latency and reliability depend on the selected backend. Finally, the reported 0% ASR should be understood only within the evaluated attack set, not as a universal security guarantee.

Q15 : What future work can be extended? (未來可以如何延伸?)

A15: Future work can extend the research in four directions. First, evaluate InteractionGuard on community benchmarks such as AgentDojo to improve comparability with prior defenses. Second, test stronger adaptive attackers that know the defense architecture. Third, extend the framework to multimodal agents where images, audio, or video can also carry injected instructions. Fourth, improve Layer 3 efficiency through better trace compression, cheaper local LLMs, or hybrid policy checking, so the system can support larger-scale real-time deployment.

Q16 : If given unlimited resources, what would you improve? (如果資源無限制，你會如何改進?)

A16: With unlimited resources, I would build a larger benchmark environment that includes real-world tools, more domains, multilingual data, adaptive attacks, and multimodal inputs. I would also train and evaluate specialized auditor models using large-scale interaction-history data, while still keeping strict separation between data and instructions. In addition, I would deploy InteractionGuard in a real agent platform to measure long-term utility, user experience, false positives, and operational cost under realistic workloads.

Q17 : Do you have any other important additions? (你有無其他重要補充?)

A17: The most important addition is the central philosophy of this work: protecting an LLM-based agent should focus on behavior, not only on input. In human terms, we cannot secure a decision-maker only by checking every document they read; we also need to observe whether their behavior still follows the original goal. InteractionGuard applies this idea to LLM agents. In the evaluated scope, the full three-layer cascade achieves balanced detection performance with $F1 = 0.9582$, low $FPR = 0.0062$, and reduces the evaluated attack success rate to 0%, showing that interaction-level defense is both effective and practical.